

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ  
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ЛЬВІВСЬКА ПОЛІТЕХНІКА



**АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ  
КОМП'ЮТЕРНИХ СИСТЕМ**

Завдання 2: “Комунікація клієнта та сервера”

Варіант 10

**Виконав:**

ст. гр. КІ-404

Костюк І.В.

**Приняв:**

Федак П.Р.

Львів – 2024

## ЗАВДАННЯ

1. Створіть просту комунікаційну схему SW (клієнт)<-> UART <-> HW (сервер).
2. Клієнт повинен надіслати повідомлення на сервер. Сервер повинен змінити повідомлення та надіслати його назад до клієнта.
3. Створіть файл YML із такими функціями:
  - а. зібрати всі бінарні файли (створити сценарії в папці ci/іf потреба);
  - б. запустити тести;
  - в. створювати артефакти з бінарними файлами та звітами про тестування;

## ТЕОРЕТИЧНИЙ МАТЕРІАЛ

UART (Universal Asynchronous Receiver-Transmitter) — це протокол, що дозволяє передавати дані асинхронно між двома пристроями, такими як комп'ютер і мікроконтролер (наприклад, Arduino). Для цього використовуються два провідники: TX для передачі та RX для прийому даних. Основною перевагою UART є те, що він не вимагає синхронізуючого сигналу, що робить його простим у налаштуванні й застосуванні.

CMake — це інструмент для автоматизації збірки, який є кросплатформним і використовується для створення конфігураційних файлів, таких як Makefile чи Ninja, необхідних для компіляції програм. Завдяки CMake легко налаштовувати збірки для різних операційних систем, що робить його універсальним рішенням для розробників.

YAML (Yet Another Markup Language) — формат даних, призначений для зберігання конфігураційних налаштувань у простій та зрозумілій структурі. YAML широко застосовується в автоматизації, зокрема для налаштувань у CI/CD системах, таких як GitHub Actions або GitLab CI, завдяки своїй легкості в читанні та написанні.

## ВИКОНАННЯ РОБОТИ

1. Створіть просту схему зв'язку SW (клієнт) <-> UART <-> HW (сервер)



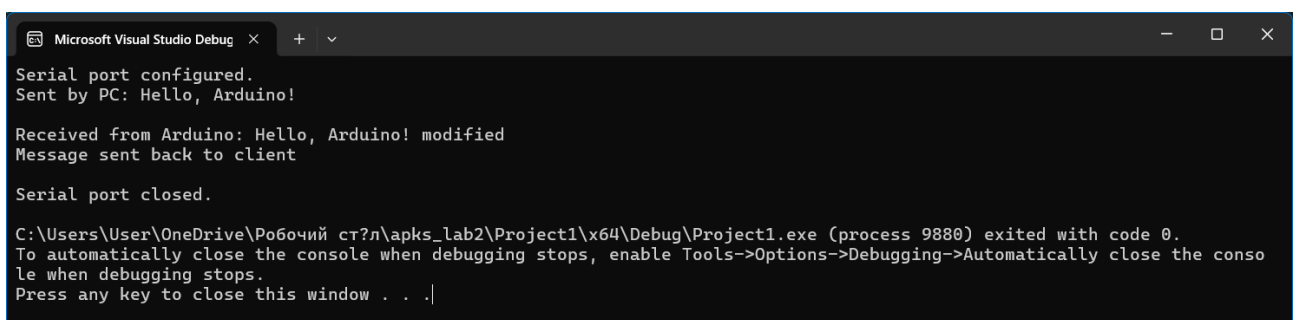
|                                                                                                                                                      |          |
|------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
|  IvanKstk Merge pull request #3 from IvanKstk/feature/develop/task2 |          |
|  .github/workflows                                                  | Task 2   |
|  client                                                             | Task 2.1 |
|  reports                                                            | Task 2   |
|  server                                                             | Task 2   |
|  CMakeLists.txt                                                     | Task 2.1 |
|  README.md                                                          | Task 2   |

*Рис.1. Структура проекту*

**Сервер** реалізовано на Arduino Mega 2560, а саме створена програму, яка приймає вхідні повідомлення, обробляє їх (змінює вміст) і відправляє змінені дані назад клієнту. Щодо **клієнта**, то у файлі `serial.h` описані оголошення функцій і змінних, необхідних для роботи з серійним портом, а у файлі `serial.cpp` реалізовані всі методи для налаштування з'єднання, передачі та прийому даних. Файл `main.cpp` є основним файлом програми, де викликаються функції для обміну повідомленнями з сервером. Тут ми надсилаємо повідомлення до Arduino і отримуємо відповідь, яку виводимо на екран.

2. Клієнт повинен надіслати повідомлення на сервер. Сервер повинен змінити повідомлення та надіслати його назад клієнту.

Після завантаження програми на Arduino та запуску коду клієнта ПК надсилає на плату повідомлення “Hello Arduino!”. Плата приймає це повідомлення, обробляє його та модифікує, додаючи до повідомлення ще «modified» і надсилає назад клієнту, після чого порт закривається.



```
Microsoft Visual Studio Debug
Serial port configured.
Sent by PC: Hello, Arduino!

Received from Arduino: Hello, Arduino! modified
Message sent back to client

Serial port closed.

C:\Users\User\OneDrive\Робочий стіл\apks_lab2\Project1\x64\Debug\Project1.exe (process 9880) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

*Рис. 2. Успішне виконання програми*

3. Створіть файл YML із такими функціями: а. зібрати всі двійкові файли (за потреби створити скрипти в папці ci/); б. запустити тести; в. створювати артефакти за допомогою бінарних файлів і звітів про тестування; назва: CI для C++ і Arduino

name: CI for C++ and Arduino Mega 2560

on:

push:

branches:

- '\*\*/\*'

pull\_request:

branches:

- '\*\*/\*'

jobs:

cpp-arduino-build:

runs-on: windows-latest

steps:

- name: Checkout code

uses: actions/checkout@v3

- name: Download Arduino CLI

uses: arduino/setup-arduino-cli@v1

with:

version: '0.19.2'

- name: Download CMake  
run: choco install cmake
- name: Download platform Arduino AVR  
run: arduino-cli core install arduino:avr
- name: Config CMake  
run: cmake -S . -B build  
shell: cmd
- name: Build with CMake  
run: cmake --build build  
shell: cmd
- name: Download build artefacts  
uses: actions/upload-artifact@v3  
with:  
  name: build-artifacts  
  path: build/\*

```

C:\WINDOWS\system32\cmd.exe
Microsoft (R) C/C++ Optimizing Compiler Version 19.39.33523 for x64
Copyright (C) Microsoft Corporation. All rights reserved.

main.cpp
C:\Program Files\Microsoft Visual Studio\2022\Community\VC\Tools\MSVC\14.39.33519\include\ostream(779): warning C4530: C++ exception handler used, but unwind semantics are not enabled. Specify /EHsc
C:\Program Files\Microsoft Visual Studio\2022\Community\VC\Tools\MSVC\14.39.33519\include\ostream(779): note: the template instantiation context (the oldest one first) is
D:\csad\client\src\main.cpp(20): note: see reference to function template instantiation 'std::basic_ostream<char,std::char_traits<char>> &std::operator <<<std::char_traits<char>>(std::basic_ostream<char,std::char_traits<char>> &,const char *)' being compiled
serial.cpp
C:\Program Files\Microsoft Visual Studio\2022\Community\VC\Tools\MSVC\14.39.33519\include\ostream(779): warning C4530: C++ exception handler used, but unwind semantics are not enabled. Specify /EHsc
C:\Program Files\Microsoft Visual Studio\2022\Community\VC\Tools\MSVC\14.39.33519\include\ostream(779): note: the template instantiation context (the oldest one first) is
D:\csad\client\src\serial.cpp(30): note: see reference to function template instantiation 'std::basic_ostream<char,std::char_traits<char>> &std::operator <<<std::char_traits<char>>(std::basic_ostream<char,std::char_traits<char>> &,const char *)' being compiled
Generating Code...
Microsoft (R) Incremental Linker Version 14.39.33523.0
Copyright (C) Microsoft Corporation. All rights reserved.

/out:client.exe
main.obj
serial.obj
[INFO] Client compiled successfully.
=====
[INFO] Compiling server code...
Sketch uses 3466 bytes (11%) of program storage space. Maximum is 30720 bytes.
Global variables use 250 bytes (12%) of dynamic memory, leaving 1798 bytes for local variables. Maximum is 2048 bytes.

Used platform Version Path
arduino:avr 1.8.6 C:\Users\User\AppData\Local\Arduino15\packages\arduino\hardware\avr\1.8.6
[INFO] Server code compiled successfully.
=====
[INFO] Uploading server code to Arduino...

```

*Рис. 3. Успішне виконання .bat файлу*

Для автоматизації етапів компіляції, лінування та генерації потрібних файлів було створено сценарій на CMake. Додатково для локальної автоматизації був розроблений .bat файл. Цей файл дозволяє користувачеві виконати всі необхідні кроки для компіляції, завантаження коду на пристрій або запуску тестів одним запуском, без потреби вводити команди вручну.

## ВИСНОВОК

У цій лабораторній роботі було реалізовано клієнт-серверну систему для простої комунікації між клієнтом (персональним комп'ютером) та сервером (Arduino) через UART-інтерфейс. Клієнт надсилає повідомлення серверу, який приймає, обробляє і модифікує отримані дані, а потім повертає їх назад, забезпечуючи двосторонній обмін інформацією.

Також була створена автоматизована система для збірки та тестування коду за допомогою YAML-конфігурації для CI/CD. Ця конфігурація автоматизує процес створення бінарних файлів, організацію скриптів у відповідній директорії та створення артефактів, що включають бінарні файли та звіти про тести. Це покращує надійність та стабільність процесу розробки завдяки безперервній інтеграції та доставці.

## СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ

1. **Arduino.** "Getting Started with Arduino." Arduino.cc.  
<https://www.arduino.cc/en/Guide/HomePage>
2. **Microsoft.** "Serial Communication in Windows." Microsoft Learn.  
<https://learn.microsoft.com/en-us/windows/desktop/devio/communications>
3. **GitHub.** "Understanding GitHub Actions." GitHub Docs.  
<https://docs.github.com/en/actions/learn-github-actions>
4. **SparkFun.** "Introduction to UART Communication." SparkFun Learn.  
<https://learn.sparkfun.com/tutorials/serial-communication/all>
5. **YAML.** "YAML Ain't Markup Language (YAML™) Specification." YAML.org.  
<https://yaml.org/>